

Build Task — Test Suite for Project 4 — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper implementation), **Advanced** (how a real project keeps this suite trustworthy over time). Read Basic first even if you already know the pattern — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building three pieces that work together: a fixed list of test tasks, a judge that scores an answer against rules, and a runner that ties it all together and prints a report.

Start with the list of tasks. Each one needs: an input to give Project 4, and a way to know if the answer was good — either an exact check (for anything that doesn't touch the model) or a set of rules for the judge to check.

Don't write any code yet. Write down, in plain English, 10 real tasks you'd actually give Project 4, and for each one, how you'd know if the answer was right.

Things to use:

- A plain Python list of dicts (or small dataclasses) for `tasks.py` — one entry per task.
- The judge pattern from this document's Real-world exercise — reuse it, don't rebuild it from scratch.
- `pytest` for the unit-layer tests that don't need a model at all.
- A simple loop in `run_eval.py` that goes through every task, runs it, scores it, and collects the results.
- `json` (built into Python) to save the report to a file, so you have a record after the run finishes.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

This Build Task combines everything from this document's Practice Exercises into one runnable thing: **exact tests** (Basic exercise) for anything with no model call, **LLM-as-judge scoring** (Real-world exercise) for final answer quality, and a **regression check** (Failure exercise) that proves the whole thing actually works.

Structure it as three separate concerns, kept in separate files: 1. **The task list** — a fixed set of at least 10 inputs, each with a clear way to check correctness, written down *before* you run anything against Project 4. 2. **The judge** — one function that scores a single output against a task's rules, with its own prompt saved somewhere versioned. 3. **The runner** — loops over every task, calls Project 4, scores the result (exact check or judge, depending on the task), and prints/saves an overall report.

The exact pieces:

- **tasks.py** — a plain list of dataclasses is enough; each one needs an `id`, an `input` (what you send to Project 4), and either `expected` (for exact checks) or `rules` (for the judge).
- **judge.py** — reuse the `build_judge_prompt()` / `score_output()` pattern from the Real-world exercise almost unchanged; the only new part is calling it from inside a loop over many tasks instead of one.
- **run_eval.py** — for each task: run Project 4 on `task.input`, decide whether it's an exact-check task or a judge task, score it accordingly, and append the result to a report list. At the end, print (or save to `report.json`) the overall pass rate and each task's individual result.
- **The unit layer (test_unit_layer.py)** — reuse the pattern from this document's Basic and Intermediate exercises (chunking, tool logic) — ordinary `pytest` tests with no LLM call, run separately/faster than the full eval suite.

Sketch the 3-file layout and what each file imports from the others, before checking Hint 2.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

A suite that runs once, by hand, on your laptop, the day you wrote it, isn't really protecting anything — it's protecting you from regressions *you remember to go check for*. The real design question this Build Task is quietly setting up for later: **how does this suite keep doing its job automatically, on every change, without you having to remember to run it?**

That reframes the runner from "a script I run sometimes" into "a gate a change has to pass through." Two things follow from that:

- **A pass/fail exit code, not just printed output.** A CI system (like GitHub Actions) doesn't read your terminal output — it reads the process's exit code. `run_eval.py` should `sys.exit(1)` when the overall score falls below your bar, and `sys.exit(0)` when it doesn't, so a CI job can be configured to block a merge on failure.
- **A saved history, not just one `report.json` that gets overwritten.** If every run overwrites the same file, you can see *that* today's score is 0.8, but not whether that's better or worse than last week — which is exactly the "quiet quality drop" this whole document is about catching. Appending each run's score (with a timestamp or commit hash) to a small history file turns one snapshot into a trend you can actually watch.

The extra pieces needed:

- `sys.exit(0)` / `sys.exit(1)` at the end of `run_eval.py`, based on `overall_score >= PASS_BAR`.
- A `history.jsonl` file (one JSON object per line, appended to — never overwritten) recording `{"timestamp": ..., "overall_score": ..., "commit": ...}` for every run, so score-over-time is a real, inspectable record.
- A minimal CI config (e.g. a GitHub Actions YAML step running `python run_eval.py`) — you don't need to write and test this for the exercise, but sketch what it would run and when (every pull request, say).

Sketch the exit-code logic and the one line that appends to `history.jsonl`, before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both describe building a suite you run yourself, once, and read the printed result. Advanced treats the suite as something that has to keep working without you — a real exit code so it can gate a merge automatically, and a score history so a slow, quiet drift downward (which no single before/after regression check would catch) becomes visible over many runs, not just the one you happen to be looking at right now.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
tasks.py:
    make a list of 10+ tasks, each with:
        an id, an input, and either an expected exact answer or a set of rules

judge.py:
    function score_with_judge(task, output):
        build a prompt with the task, output, and rules
        call the model, parse PASS/FAIL and the reason
        return the result

run_eval.py:
    function run_eval_suite():
        results = []
        for each task in tasks:
            output = run Project 4 on task.input
            if task has an exact "expected" answer:
                result = check output == expected (or a simple property)
            else:
                result = score_with_judge(task, output)
            add result to results
        print a report: how many passed, which ones failed and why
        return results
```

Here's almost the whole thing — the trickiest part, deciding whether a task needs an exact check or the judge, in one function:

```
def run_one_task(task, project_4_pipeline, score_with_judge):
    output = project_4_pipeline(task["input"])

    if "expected" in task:
        passed = task["expected"].lower() in output.lower()
        reason = "exact/substring check"
    else:
        result = score_with_judge(task["input"], output, task["rules"])
        passed = result["verdict"] == "PASS"
        reason = result["reason"]
```

```
return {"id": task["id"], "passed": passed, "reason": reason}
```

Now write `run_eval_suite()` , which loops over all tasks using this function and builds the final report.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
tasks.py:
    define Task: id, input, expected (optional), rules (optional)
    build TASKS: a list of 10+ Task entries, mixing exact-check and rule-based ta

judge.py:
    define JudgeResult: verdict, reason
    function build_judge_prompt(task_input, output, rules):
        combine input, output, rules into one prompt asking for PASS/FAIL + reason
    function score_with_judge(task_input, output, rules) -> JudgeResult:
        call the model with build_judge_prompt(...)
        parse the reply
        return JudgeResult

run_eval.py:
    function run_one_task(task) -> TaskResult:
        output = call project_4_pipeline(task.input)
        if task.expected is set:
            passed = (a property check against task.expected, not necessarily ==)
            reason = "exact check"
        else:
            judge_result = score_with_judge(task.input, output, task.rules)
            passed = judge_result.verdict == "PASS"
            reason = judge_result.reason
        return TaskResult(task.id, passed, reason)

    function run_eval_suite() -> Report:
        results = [run_one_task(task) for task in TASKS]
        overall_score = count of passed / total
        save results + overall_score to report.json
        print a summary
        return Report(results, overall_score)

    if this file is run directly: call run_eval_suite()
```

```
from dataclasses import dataclass
from typing import Optional
```

```
@dataclass
```

```

class TaskResult:
    task_id: str
    passed: bool
    reason: str

def run_one_task(task: dict, project_4_pipeline, score_with_judge) -> TaskResult:
    output = project_4_pipeline(task["input"])

    if "expected" in task:
        passed = task["expected"].lower() in output.lower()
        reason = "exact/substring check"
    else:
        result = score_with_judge(task["input"], output, task["rules"])
        passed = result.verdict == "PASS"
        reason = result.reason

    return TaskResult(task_id=task["id"], passed=passed, reason=reason)

```

What's missing: `run_eval_suite()`, which loops over `TASKS`, calls `run_one_task` for each, and builds/prints/saves the overall report — plus the regression-check script from the Failure exercise, run against this same suite. Write both yourself before moving to Advanced.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

run_eval.py, extended:

```
function run_eval_suite() -> Report:
  (same as Intermediate: build results, overall_score)
  append {timestamp, commit, overall_score} as one line to history.jsonl
  write report.json as before
  print summary
  return Report
```

at the bottom of the file:

```
if overall_score < PASS_BAR:
  print a clear failure message
  sys.exit(1)  # <- this is what lets a CI job block on it
else:
  sys.exit(0)
```

Here's almost the whole thing — fill in the history-append line yourself:

```
import json
import subprocess
import sys
import time
from dataclasses import asdict

PASS_BAR = 0.8

def get_git_commit() -> str:
  try:
    return subprocess.check_output(["git", "rev-parse", "--short", "HEAD"]).d
  except Exception:
    return "unknown"

def run_eval_suite(project_4_pipeline, call_model) -> dict:
  results = [run_one_task(task, project_4_pipeline, call_model) for task in TAS
  passed_count = sum(1 for r in results if r.passed)
  overall_score = passed_count / len(results)
```

```

report = {
    "overall_score": overall_score,
    "passed": passed_count,
    "total": len(results),
    "results": [asdict(r) for r in results],
}
with open("report.json", "w") as f:
    json.dump(report, f, indent=2)

# your turn: append one line to history.jsonl with timestamp, commit, and over
...

print(f"Passed {passed_count}/{len(results)} (score: {overall_score:.2f})")
return report

if __name__ == "__main__":
    result = run_eval_suite(project_4_pipeline=my_pipeline, call_model=my_call_mo
    if result["overall_score"] < PASS_BAR:
        print(f"FAILED: score {result['overall_score']:.2f} is below the bar of {
        sys.exit(1)
    sys.exit(0)

```

Fill in the `history.jsonl` append (one JSON object per line, opened in append mode `"a"`), then compare all 3 of your finished versions against the Solution below.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both build a suite that runs and reports once. Advanced adds the 2 pieces from Hint 1's Advanced question made real: a `sys.exit()` code so the run can gate a CI pipeline, and an append-only `history.jsonl` so score-over-time becomes a real record instead of a single overwritten snapshot — the difference between "I ran the suite once" and "this suite protects every future change automatically."

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Read all three depths — they're not "wrong, less wrong, right," they're 3 real, valid ways to build the same suite, with real tradeoffs between them.

Basic Version

Approach 1 — a plain list of dicts, one report file

```
# tasks.py
TASKS = [
    {"id": "t1", "input": "What's your return policy?", "expected": "30 days"},
    {"id": "t2", "input": "Summarize this ticket", "rules": ["mentions the correc
    # ... 8+ more tasks
]
```

```
# judge.py
def build_judge_prompt(task_input, output, rules):
    rules_text = "\n".join("- " + r for r in rules)
    return (
        "Task: " + task_input + "\n"
        "Answer: " + output + "\n"
        "Rules:\n" + rules_text + "\n"
        "Reply PASS or FAIL on the first line, then why."
    )

def score_with_judge(task_input, output, rules, call_model):
    prompt = build_judge_prompt(task_input, output, rules)
    response = call_model(prompt)
    lines = response.strip().split("\n", 1)
    return {"verdict": lines[0].strip().upper(), "reason": lines[1] if len(lines)
```

```
# run_eval.py
import json
from tasks import TASKS
from judge import score_with_judge

def run_eval_suite(project_4_pipeline, call_model):
    results = []
```

```

for task in TASKS:
    output = project_4_pipeline(task["input"])
    if "expected" in task:
        passed = task["expected"].lower() in output.lower()
        reason = "exact/substring check"
    else:
        judged = score_with_judge(task["input"], output, task["rules"], call_
        passed = judged["verdict"] == "PASS"
        reason = judged["reason"]
    results.append({"id": task["id"], "passed": passed, "reason": reason})

passed_count = sum(1 for r in results if r["passed"])
report = {"passed": passed_count, "total": len(results), "results": results}

with open("report.json", "w") as f:
    json.dump(report, f, indent=2)

print("Passed " + str(passed_count) + "/" + str(len(results)))
return report

if __name__ == "__main__":
    run_eval_suite(project_4_pipeline=my_pipeline, call_model=my_call_model)

```

Keep `test_unit_layer.py` as normal `pytest` tests (no model, no report file — just pass/fail in the terminal), completely separate from `run_eval.py`'s model-touching suite. Run them as two separate commands: `pytest test_unit_layer.py` for the fast, always-the-same layer, and `python run_eval.py` for the full eval report.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — dataclasses throughout, one runner script

tasks.py

```
from dataclasses import dataclass
from typing import Optional

@dataclass
class Task:
    id: str
    input: str
    expected: Optional[str] = None
    rules: Optional[list[str]] = None

TASKS: list[Task] = [
    Task(id="t1", input="What's your return policy?", expected="30 days"),
    Task(
        id="t2",
        input="Summarize this support ticket about a late delivery.",
        rules=["mentions the correct category", "is under 100 words", "does not i
    ),
    # ... 8+ more tasks, mixing exact-check and rule-based
]
```

judge.py

```
from dataclasses import dataclass

@dataclass
class JudgeResult:
    verdict: str
    reason: str

JUDGE_PROMPT_VERSION = "v1"
```

```
def build_judge_prompt(task_input: str, output: str, rules: list[str]) -> str:
    rules_text = "\n".join(f"- {r}" for r in rules)
    return (
        f"Task: {task_input}\n"
        f"Answer to check: {output}\n"
        f"Rules the answer must follow:\n{rules_text}\n"
        "Reply with PASS or FAIL on the first line, then one sentence explaining
    )

def score_with_judge(task_input: str, output: str, rules: list[str], call_model)
    prompt = build_judge_prompt(task_input, output, rules)
    response = call_model(prompt)
    lines = response.strip().split("\n", 1)
    verdict = lines[0].strip().upper()
    reason = lines[1].strip() if len(lines) > 1 else ""
    return JudgeResult(verdict=verdict, reason=reason)
```

run_eval.py

```
import json
from dataclasses import dataclass, asdict
from tasks import TASKS, Task
from judge import score_with_judge

@dataclass
class TaskResult:
    task_id: str
    passed: bool
    reason: str

def run_one_task(task: Task, project_4_pipeline, call_model) -> TaskResult:
    output = project_4_pipeline(task.input)

    if task.expected is not None:
        passed = task.expected.lower() in output.lower()
        reason = "exact/substring check"
    else:
        judged = score_with_judge(task.input, output, task.rules, call_model)
        passed = judged.verdict == "PASS"
        reason = judged.reason

    return TaskResult(task_id=task.id, passed=passed, reason=reason)
```

```

def run_eval_suite(project_4_pipeline, call_model) -> dict:
    results = [run_one_task(task, project_4_pipeline, call_model) for task in TASKS]
    passed_count = sum(1 for r in results if r.passed)
    overall_score = passed_count / len(results)

    report = {
        "overall_score": overall_score,
        "passed": passed_count,
        "total": len(results),
        "results": [asdict(r) for r in results],
    }

    with open("report.json", "w") as f:
        json.dump(report, f, indent=2)

    print(f"Passed {passed_count}/{len(results)} (score: {overall_score:.2f})")
    for r in results:
        if not r.passed:
            print(f"  FAILED {r.task_id}: {r.reason}")

    return report

if __name__ == "__main__":
    run_eval_suite(project_4_pipeline=my_pipeline, call_model=my_call_model)

```

Approach 2 — a `pytest` -native suite, using `parametrize` for the eval tasks too

```

# test_eval_suite.py
import pytest
from tasks import TASKS
from judge import score_with_judge

@pytest.mark.parametrize("task", TASKS, ids=lambda t: t.id)
def test_task(task, project_4_pipeline, call_model):
    output = project_4_pipeline(task.input)

    if task.expected is not None:
        assert task.expected.lower() in output.lower()
    else:
        result = score_with_judge(task.input, output, task.rules, call_model)
        assert result.verdict == "PASS", result.reason

```

Difference from Basic: dataclasses everywhere give you typed, autocomplete-friendly objects at every layer. Printing failed task reasons directly (not just the count) means a failing run tells you exactly where to look, without opening `report.json` first. Approach 2 runs both layers through one `pytest -v` command with pytest's own reporting, trading away the single `report.json` with an overall score unless you add a small script that also reads pytest's own JSON report output.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — exit code + append-only score history, so the suite can gate CI

`run_eval.py`, extending the Intermediate version:

```
import json
import subprocess
import sys
import time
from dataclasses import asdict
from tasks import TASKS, Task
from judge import score_with_judge

PASS_BAR = 0.8

def get_git_commit() -> str:
    try:
        return subprocess.check_output(["git", "rev-parse", "--short", "HEAD"]).d
    except Exception:
        return "unknown"

def run_one_task(task: Task, project_4_pipeline, call_model):
    output = project_4_pipeline(task.input)
    if task.expected is not None:
        passed = task.expected.lower() in output.lower()
        reason = "exact/substring check"
    else:
        judged = score_with_judge(task.input, output, task.rules, call_model)
        passed = judged.verdict == "PASS"
        reason = judged.reason
    return {"task_id": task.id, "passed": passed, "reason": reason}

def run_eval_suite(project_4_pipeline, call_model) -> dict:
    results = [run_one_task(task, project_4_pipeline, call_model) for task in TASKS]
    passed_count = sum(1 for r in results if r["passed"])
    overall_score = passed_count / len(results)

    report = {
```

```

        "overall_score": overall_score,
        "passed": passed_count,
        "total": len(results),
        "results": results,
    }
    with open("report.json", "w") as f:
        json.dump(report, f, indent=2)

    history_entry = {
        "timestamp": time.time(),
        "commit": get_git_commit(),
        "overall_score": overall_score,
    }
    with open("history.jsonl", "a") as f:
        f.write(json.dumps(history_entry) + "\n")

    print(f"Passed {passed_count}/{len(results)} (score: {overall_score:.2f})")
    for r in results:
        if not r["passed"]:
            print(f"  FAILED {r['task_id']}: {r['reason']}")

    return report

if __name__ == "__main__":
    result = run_eval_suite(project_4_pipeline=my_pipeline, call_model=my_call_model)
    if result["overall_score"] < PASS_BAR:
        print(f"FAILED: score {result['overall_score']:.2f} is below the bar of {PASS_BAR:.2f}")
        sys.exit(1)
    sys.exit(0)

```

Expected output on a passing run: the same summary as Intermediate, plus one new line appended to `history.jsonl` each time (`{"timestamp": 1234567.0, "commit": "a1b2c3d", "overall_score": 0.9}`), and the process exits with code `0` — check with `echo $?` right after running it.

Expected behavior on a failing run: the same report and history line are written (you still want a record of the bad run), then `FAILED: score 0.60 is below the bar of 0.8` prints and the process exits with code `1` — `echo $?` shows `1`, which is exactly what a CI step checks to decide whether to block a merge.

A minimal GitHub Actions step that uses this:

```

# .github/workflows/eval.yml (illustrative – not required for this exercise)
- name: Run eval suite

```

```
run: python run_eval.py
```

No extra CI-specific code needed — the job simply fails when `run_eval.py` exits non-zero.

Approach 2 — reading `history.jsonl` back to show a trend, not just today's number

```
import json

def load_score_history(path: str = "history.jsonl") -> list[dict]:
    entries = []
    with open(path) as f:
        for line in f:
            if line.strip():
                entries.append(json.loads(line))
    return entries

def print_trend(path: str = "history.jsonl", last_n: int = 10) -> None:
    entries = load_score_history(path)[-last_n:]
    for entry in entries:
        commit = entry["commit"]
        score = entry["overall_score"]
        bar = "#" * int(score * 20)
        print(f"{commit:>10} {score:.2f} {bar}")

if __name__ == "__main__":
    print_trend()
```

Expected output:

```
alb2c3d  0.90 #####
d4e5f6a  0.90 #####
g7h8i9j  0.60 #####
k1l2m3n  0.85 #####
```

That one lower row (`g7h8i9j` , 0.60) is exactly the kind of quiet quality drop this document's Core Concepts warns about — visible here as a dip in a trend, where a single "did today's run pass" check would only tell you about the most recent commit, not that things got briefly worse and then partly recovered.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's `run_eval_suite()` is a script you run and read — correct, but nothing stops a real regression from

shipping if nobody happens to run it that day. Approach 1 turns the same function into something a CI system can act on automatically, via a real exit code, and starts keeping a permanent, append-only record instead of one overwritten `report.json`. Approach 2 is the other half of that record's value: reading `history.jsonl` back as a trend, which is the only way to notice a *gradual* drift (score slowly creeping down over several small prompt tweaks) that no single before/after regression check, including this document's own Failure exercise, is designed to catch on its own.

Which one should you actually use? Intermediate Approach 1 is what to build and use day-to-day while working on Project 4 yourself — it's everything the Build Task's stated requirements ask for. Add Advanced Approach 1's exit code and history file the moment this suite needs to run somewhere other than your own terminal — a CI pipeline, a teammate's machine, a scheduled job — since a script that only "works" when a human reads its printed output isn't actually gating anything. Add Approach 2's trend view once you have more than a handful of history entries worth looking at; before that, `report.json` from a single run is enough.